

2 주차 진행 공유

시험 벼락치기 스케줄러

시험기간 대학생들을 위한 수면 · 카페인 스케줄 최적화 도구 — 계산 엔진 파트

N110_ 안정연

무엇을, 왜 만드는가

상황

시험기간이 되면 대학생 대부분은 벼락치기를 하고, 수면 · 카페인 섭취를 순전히 감에 의존해 조절한다. 그 결과 수면 부족 · 컨디션 저하가 누적되고, 시험이 여러 개 겹치는 주간 전체는 고려하지 못한 채 "오늘 밤"만 생각하게 된다.

만드는 것

시험 날짜 · 공부량 · 수면패턴 · 카페인 상태를 입력하면, 검증된 생물수학 모델로 "시험 시작 시각에 각성도가 최고가 되는" 수면 · 카페인 스케줄을 계산해주는 도구.

핵심 기능

하루 단위 스케줄이 아니라, 시험이 여러 개 겹치는 "시험기간 전체"를 한 번에 고려해 수면 · 카페인 스케줄을 배분해준다.

사용한 모델

Two-Process Model(수면압 + 일주기리듬) + 카페인 약동학 · 약리학(PK/PD) 상호작용 모델
— 수면과학 논문 기반의 검증된 공식 사용

프로젝트 로드맵 (4 주)

1 주차

- 아이디어 검증 및 문제 정의
- 기획서 작성 (v1 → v2)
- 디자인 문서 · 스킬 설정
- 프로토타입 제작 → React 이식

완료

2 주차

- Process S/C(수면압 · 일주기) 구현
- 카페인 PK/PD 근사 모델 구현
- 안전 섭취 한도 로직
- 목표 각성 시각 역산 (그리드 탐색)

진행 중

3 주차

- 다중 시험 통합 최적화
- Supabase 연동
- 화면에 계산 기능 실제 연결

다음 순서

4 주차

- 추가 기능 (여유 시)
- 전체 테스트 · 버그 수정
- 발표 자료 · 데모 준비

예정

지금 여기 : 2 주차 — 화면 없이도 계산 로직이 순수 함수로 끝까지 완성된 상태를 목표로 함

2 주차 목표 · 진행 상황 (07/13 ~ 07/16)

목표 : 화면 없이도 " 입력 → 시험 시작 시각에 각성도가 최고가 되는 스케줄 " 까지 순수 함수로 계산되게 만들기

완료 (2 주차 범위)

- ✓ Express 서버 스캐폴딩
- ✓ Process S(수면압) · Process C(일주기리듬)
- ✓ 수면관성 · 오후슬럼프 보정
- ✓ 카페인 근사 모델 (PK + PD + 결합)
- ✓ 안전 섭취 한도 로직
- ✓ 목표 각성 시각 역산 (시험 시작 시각 그대로 + 후보 그리드 탐색)

3 주차로 이동



다중 시험 통합 최적화
(그리드 결정변수 정의 — #10)



Supabase 프로젝트 생성 및 마이그레이션
(#14)

All issues

[New issue](#)

is:issue state:closed



☐	Open	11	Closed	9	Author ▾	Labels ▾	Projects ▾	Milestones ▾	Assignees ▾	Types ▾	⌵ Newest ▾
☐	✓	안전 섭취 한도 로직	BE	priority:필수							1
#9 · by dodeho was closed 1d ago · 2주차											
☐	✓	목표 작성 시각 역산 (단순 탐색, 시험 1개)	BE	priority:중요							1
#8 · by dodeho was closed 2h ago · 2주차											
☐	✓	$P_0 \times \text{카페인효과} = P(t)$ 최종 결합	BE	priority:필수							1
#7 · by dodeho was closed 1d ago · 2주차											
☐	✓	카페인 효과 함수 구현	BE	priority:필수							1
#6 · by dodeho was closed 1d ago · 2주차											
☐	✓	S+C 결합(P_0) 및 검증	BE	priority:중요							1
#5 · by dodeho was closed 2d ago · 2주차											
☐	✓	Process C(일주기리듬) 함수 구현	BE	priority:필수							
#4 · by dodeho was closed 2d ago · 2주차											
☐	✓	Process S(수면압) 함수 구현	BE	priority:필수							
#3 · by dodeho was closed 2d ago · 2주차											
☐	✓	카페인 근사 함수 파라미터 확정 (문서화)	BE	priority:필수							
#2 · by dodeho was closed 3d ago · 2주차											
☐	✓	Express 서버 스케폴딩	BE	priority:필수							
#1 · by dodeho was closed 3d ago · 2주차											

기술 스택

서비스 기술 지도



FE — React

입력 폼 · 캘린더 UI, 각성도 그래프 · 타임라인 시각화

5 개 화면 모두 구현됐지만
아직 mock 데이터로 동작



BE — Express

2 주차에 계산 엔진 (Two-Process+ 카페인)
순수 함수를 완성.
아직 화면과 연결하는 API 는 3 주차 작업



DB — Supabase (Postgres)

카페인 함량표 · 반감기 · 안전 섭취 한도 등
참고용 정적 데이터 저장 — 테이블 설계만 완료

계산 엔진 — server/src/calc/

processS · processC · sleepInertia · lunchDip · caffeineConcentration · caffeineEffect · alertness ·
dailyCaffeineLimit · candidateSleepSegments · singleExamScheduleSearch (총 11 개 순수 함수 파일)

프론트엔드

React 프로토타입 소개



홈 → 정보 입력 → 처리 중 → 결과 → 스케줄 조정

5 개 화면이 실제로 라우팅까지 연결된 동작하는 React 앱 (client/)

React 19

TypeScript

Vite

react-router-dom

CSS Modules

로컬 개발 서버

`http://localhost:5173/` (**`npm run dev --prefix client`**)

아직 배포 전 : GitHub Pages(dodeho.github.io/hub/) 엔 예전 정적 HTML 목업이 그대로 떠 있다 — React 앱 배포는 화면 · 계산 엔진이 연결되는 3 주차에 맞출 예정

React 컴포넌트 & 폴더 구성

구현된 컴포넌트 — client/src/components/

```
client/src/components/  
├─ BottomSheet/ (BottomSheet.tsx + .module.css)  
├─ Button/  
├─ Card/  
├─ Field/  
├─ Row/  
├─ Segmented/  
├─ Slider/  
├─ Switch/  
├─ WarningBanner/  
├─ icons.tsx  
└─ index.ts (모아서 export)
```

9 개 컴포넌트 모두 *PascalCase.tsx* +
동일 이름 *.module.css* 짝으로 구성됨 (*CLAUDE.md* 컨벤션)

페이지 & 그 외 구성 — client/src

pages/

Home · Input · Processing · Result · Adjust (5 개)

layouts/AppShell

공통 화면 뼈대 (헤더 · 진행바 등)

styles/tokens.css

디자인 토큰 (컬러 · 타이포그래피)

utils/time.ts

시각 표시 · 변환 유틸 함수

5 개 페이지가 실제로 라우팅까지 연결된
동작하는 앱 (*react-router-dom*)


```

1 import type { ReactNode } from 'react';
2 import { Link } from 'react-router-dom';
3 import styles from './Button.module.css';
4
5 interface ButtonProps {
6   children: ReactNode;
7   variant?: 'primary' | 'secondary';
8   size?: 'default' | 'sm';
9   to?: string;
10  onClick?: () => void;
11  type?: 'button' | 'submit';
12 }
13
14 /** docs/디자인.md 5번 "버튼" 이 to가 있으면 라우터 링크, 없으면 버튼 */
15 export function Button({ children, variant = 'primary', size = 'default', to, onClick, type = 'button' }: Butt
16   const className = [
17     styles.btn,
18     variant === 'primary' ? styles.primary : styles.secondary,
19     size === 'sm' ? styles.sm : '',
20   ]
21     .filter(Boolean)
22     .join(' ');
23
24   if (to) {
25     return (
26       <Link to={to} className={className}>
27         {children}
28       </Link>
29     );
30   }
31
32   return (
33     <button type={type} className={className} onClick={onClick}>
34       {children}
35     </button>
36   );
37 }

```

```
1  .btn {
2    display: inline-flex;
3    align-items: center;
4    justify-content: center;
5    gap: 6px;
6    padding: 13px 18px;
7    border-radius: 15px;
8    font-weight: 700;
9    font-size: 14.5px;
10   text-decoration: none;
11   border: 1px solid transparent;
12   cursor: pointer;
13   flex: 1;
14   font-family: var(--font);
15 }
16
17 .primary {
18   background: var(--brand);
19   color: #fff;
20 }
21
22 .secondary {
23   background: var(--surface-muted);
24   color: var(--ink-900);
25 }
26
27 .sm {
28   flex: none;
29   padding: 8px 16px;
30   font-size: 13px;
31   border-radius: 11px;
32 }
```

어떻게 계산할까 — 모델 개념



졸음은 쌓인다 (Process S)

깨어있는 시간이 길수록 졸음 (수면압) 이 쌓이고, 자는 동안 풀린다



생체시계가 있다 (Process C)

하루 주기로 오르내리는 원래 정해진 각성 리듬이 따로 있다



카페인을 일시적으로 억제한다

카페인 졸음 신호를 일시적으로 눌러줘서, 원하는 시각에 각성도를 끌어올릴 수 있다

2 주차에 세 개를 전부 순수 함수로 구현하고, 결합 · 검증까지 마쳤다

결합 공식 : $P(t) = P_0(t) \times gPD(t)$ — 수면압 + 일주기리듬으로 만든 기본 각성도 (P_0) 에 , 카페인 보정 계수 (gPD) 를 곱해 최종 각성도를 구한다

어떻게 최적 스케줄을 찾을까 — 계산 과정 3 단계



① 시간 그리드 만들기

오늘부터 마지막 시험까지 30 분 단위로 쪼개,
깨어있기 / 잠자기 × 카페인 섭취 / 미선택로
나눔



② 후보마다 시뮬레이션

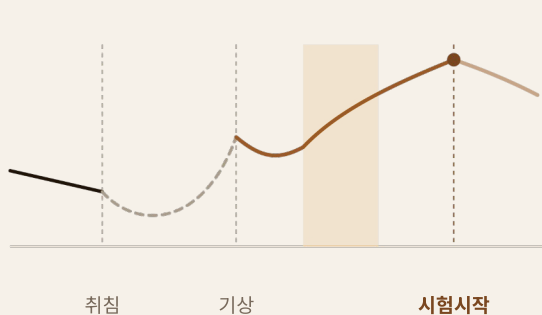
각 조합을 모델에 대입해 각성도 곡선을 계산,
시험 시각 각성도엔 가산점, 제약 위반엔 패널티



③ 최고점 스케줄 채택

전수탐색이 아니라 후보를 반복 개선하는 국소
탐색,
점수가 가장 높은 조합을 추천 스케줄로 제공

예시 : 이 과정을 거쳐 선택된 스케줄의 예측 각성도 곡선



■ 깨어있는 동안 예측 각성도

■ 수면 중 구간 (모델상 회복)

■ 카페인 부스트 구간

■ 정점 = 시험 시작 시각

Process S — 수면압



processS.ts

깨어있는 동안 : $H(t) = \mu + (H_0 - \mu) \cdot e^{-(t-t_0)/\chi_w}$

자는 동안 : $H(t) = H_0 \cdot e^{-(t-t_0)/\chi_s}$

$\chi_s = 4.2h$ (방전), $\chi_w = 18.2h$ (충전), $\mu = 1$ (상한)

왜 이렇게 작성했나

계산 _ 모델 _ 리서치 .md 1 장 수식을 그대로 옮겼다 .

χ_s (방전 4.2h) · χ_w (충전 18.2h) · μ (상한 1) 는 Two-Process Model 표준값 (Borbély 1982, Daan · Beersma · Borbély 1984) 을 그대로 썼다 .

— 이 함수는 임의로 튜닝한 값이 하나도 없는 구간이다 .

```
server > src > calc > TS processS.ts > ...
```

```
1  // 계산 모델 리서치.md 1장 "Process S(수면압)" 수식 그대로 구현
2  const CHI_SLEEP = 4.2; // 자는 동안 방전 속도(시간)
3  const CHI_WAKE = 18.2; // 깨어있는 동안 충전 속도(시간)
4  const CEILING = 1; // 1, 충전 상한선
5
6  export interface SleepPressureSegment {
7    /** 이번 구간(수면 또는 각성)이 시작된 시각. 자정 기준 경과 시간(0~24+) */
8    startTime: number;
9    /** 구간 시작 시점의 피로도( $H_0$ ) */
10   startPressure: number;
11   isAsleep: boolean;
12 }
13
14 export function sleepPressure(t: number, segment: SleepPressureSegment): number {
15   const { startTime, startPressure, isAsleep } = segment;
16   const elapsed = startTime - t;
17
18   if (isAsleep) {
19     return startPressure * Math.exp(elapsed / CHI_SLEEP);
20   }
21   return CEILING + (startPressure - CEILING) * Math.exp(elapsed / CHI_WAKE);
22 }
```

Process C — 일주기리듬



processC.ts

$C(t) = \sum a_n \cdot \sin(2\pi n \cdot t/24 + \Phi)$ (n = 1~5, 5-하모닉)

$a = [0.97, 0.22, 0.07, 0.03, 0.001]$

왜 이렇게 작성했나

일주기 리듬은 사인파 하나로는 실제 곡선과 잘 안 맞아서, 진폭이 다른 5 개 하모닉을 그대로 더하는 표준형을 썼다.

위상 (Φ) 은 논문에 고정값이 없어서, 최저점이 새벽 4 시경에 오도록 5 개 하모닉을 다 더한 채로 직접 탐색해서 맞춘 값이다.

— 따라서, 위상 값이 과학적 근거가 있다고 장담할 수 없다.

TS processC.ts ✕

server > src > calc > TS processC.ts > ...

```
1 // 계산 모델 리서치.md 1장 "Process C(일주기리듬)" (b) 5-하모닉 수식 그대로 구현
2 const HARMONIC_AMPLITUDES = [0.97, 0.22, 0.07, 0.03, 0.001]; // a1~a5
3 const TAU = 24; // 하루 주기(시간)
4
5 // 0: 최저점이 새벽 4시경 오도록 맞춘 위상값. 논문에 고정값 없음 || 원래 각자 조정해서 쓰는 값(2026-07-13 확인).
6 // a1(기본음)만 고려한 계산은 최저점이 2.4시로 어긋나서, 5개 하모닉을 다 더한 상태로 직접 탐색해서 맞춤
7 const PHASE = -Math.PI;
8
9 export function circadianRhythm(t: number): number {
10   return HARMONIC_AMPLITUDES.reduce((sum, amplitude, index) => {
11     const harmonicNumber = index + 1;
12     return sum + amplitude * Math.sin((2 * Math.PI * harmonicNumber * t) / TAU + PHASE);
13   }, 0);
14 }
```


수면관성 · 오후슬럼프 보정



sleepInertia.ts · lunchDip.ts

수면관성 : $-0.3 \cdot e^{-(\text{경과시간} / 0.67h)}$ (기상 직후만)

오후슬럼프 : $-0.15 \cdot e^{-(\text{시각} - 14)^2 / 2}$ (매일 14 시 중심)

왜 이렇게 작성했나

S+C 만 더해서 곡선을 뽑아보니 기상 직후 각성도가 비현실적으로 높게 나왔다 .

그리고 오후슬럼프도 과학적으로 여러 연구가 이루어지고 있는데 고려하지 않았다는 생각이 들었다 .

— 검증 스크립트로 눈으로 확인하다가 발견한 누락이라 , 원 수식엔 없던 두 보정 항을 추가로 반영했다 .

계수 (0.3·0.15 등) 는 대부분 근거 없는 근사치이고 , 수면관성 시간상수 (0.67h) 만 Jewett & Kronauer(1999) 값을 썼다 .

근사치는 여러 기사 자료를 참고해 설정한 값이다 .

```
server > src > calc > TS sleepInertia.ts > ...
```

```
1 // 수면 관성(sleep inertia): 기상 직후 일시적으로 각성도가 떨어졌다가 서서히 회복되는 현상.  
2 // 계산 모델 리서치.md 원 수식(1장)엔 없던 항 - 검증 중 발견한 누락을 보완(2026-07-13 결정)  
3 import type { SleepPressureSegment } from "./processS.js";  
4  
5 const MAGNITUDE = 0.3; // 기상 직후 최대로 깎이는 정도. 근거 없는 근사치  
6 const TAU = 0.67; // 시간상수(시간). 출처: Jewett & Kronauer(1999), 주관적 각성도 기준  
7  
8 export function sleepInertia(t: number, segment: SleepPressureSegment): number {  
9   if (segment.isAsleep) return 0;  
10  
11   const elapsedSinceWake = t - segment.startTime;  
12   if (elapsedSinceWake < 0) return 0;  
13  
14   return -MAGNITUDE * Math.exp(-elapsedSinceWake / TAU);  
15 }
```

```
server > src > calc > TS lunchDip.ts > ...
```

```
1 // 오후 슬럼프(post-lunch dip): 점심 여부와 무관하게 이른 오후에 각성도가 살짝 처지는 현상.  
2 // 계산 모델 리서치.md 원 수식(1장)엔 없던 항 - 검증 중 발견한 누락을 보완(2026-07-13 결정)  
3 const CENTER = 14; // 답의 중심 시각(14시)  
4 const DEPTH = 0.15; // 최대로 파이는 정도. 근거 없는 근사치  
5 const WIDTH = 1; // 답의 폭(시간). 근거 없는 근사치  
6  
7 export function lunchDip(t: number): number {  
8   const hourOfDay = t % 24; // t가 24를 넘어가는(다음날 이후) 시각이어도 "하루 중 몇 시"로 환산  
9   const hoursFromCenter = hourOfDay - CENTER;  
10  return -DEPTH * Math.exp(-(hoursFromCenter ** 2) / (2 * WIDTH ** 2));  
11 }
```

카페인 PK — 혈중농도



caffeineConcentration.ts

$$C(t) = F \cdot \text{Dose} \cdot k_a / (V_d \cdot (k_a - k_e)) \cdot (e^{(-k_e \cdot \Delta t)} - e^{(-k_a \cdot \Delta t)})$$

$$F = 1, k_a = 5/h, V_d = 0.7 \text{ L/kg}$$

왜 이렇게 작성했나

경구 섭취 표준 1 차 흡수 / 제거 약동학 (PK) 모델이다 .

F(생체이용률)=1, Vd=0.7L/kg 는 공개 자료 (NCBI) 기준값을 그대로 썼고 ,

k_a(흡수속도)=5 는 음료 기준 최고혈중농도 도달 시간 (39~42 분) 을 거꾸로 계산해서 역산한 값이다 .

해당 역산은 자연로그를 이용했는데 아직 이해가 부족해서 원리에 대해서는 설명하지 못한다 .

server > src > calc > ts caffeineConcentration.ts > ...

```
1 // 계산 모델 리서치.md 2.1 "약동학(PK) - 혈중 카페인 농도"
2 // 섭취 1건에 대한 혈중농도만 계산하는 순수함수. 하루에 여러 번 섭취했을 때 농도를 합산(중첩)하는 건
3 // 이 함수를 호출하는 쪽(alertness.ts)의 책임으로 둔다 - 약동학적으로 여러 섭취의 농도는 그냥 더하면 되기 때문.
4 const F = 1; // 생체이용률. 경구 카페인은 거의 완전 흡수(~99%)됨
5 const KA = 5; // 흡수 속도 상수(/h). 음료 기준 최고 혈중농도 도달 시간(39~42분)을 역산해서 도출
6 const VD_PER_KG = 0.7; // 분포용적(L/kg). NCBI Bookshelf 자료 기준
7
8 export interface CaffeineDose {
9   /** 섭취 시각. 자정 기준 경과 시간(0~24+) */
10   time: number;
11   /** 섭취량(mg) */
12   amountMg: number;
13 }
14
15 export function caffeineConcentration(
16   t: number,
17   dose: CaffeineDose,
18   bodyWeightKg: number,
19   halfLifeHours: number,
20 ): number {
21   const elapsed = t - dose.time;
22   if (elapsed < 0) return 0; // 섭취 전엔 혈중농도 0
23
24   const vd = bodyWeightKg * VD_PER_KG;
25   const ke = Math.log(2) / halfLifeHours;
26
27   return (
28     (F * dose.amountMg * KA) /
29     (vd * (KA - ke)) *
30     (Math.exp(-ke * elapsed) - Math.exp(-KA * elapsed))
31   );
32 }
```

카페인 PD — 각성도 보정



caffeineEffect.ts

$$gPD(t) = 1 + Gmax \cdot C(t)^n / (EC50^n + C(t)^n) \quad (n = 1)$$

$Gmax = 0.05 \sim 0.25$ (피로도에 따라 가변), $EC50 = 2mg/L$

왜 이렇게 작성했나

논문에서 정확한 Hill 계수를 못 구해서 , 공개된 표준 포화형 용량 - 반응 곡선 (Hill/Michaelis-Menten) 으로 근사했다 .

$Gmax$ (최대 부스트) 는 코드에 ' 근거 없는 근사치 ' 라고 명시해뒀고 ,
 $EC50=2mg/L$ 은 200mg 섭취 시 최대혈중농도의 절반 지점으로 잡았다 .

원래는 카페인 의 각성 정도만 고려할 예정이었으나 , 피로도에 따라서 카페인 의 각성 능력이 달라질 수 있는 점을 고려해
수면압 함수를 import 해 계산된다 . (과학적 근거 부족)

server > src > calc > TS caffeineEffect.ts > ...

```
1 // 계산 모델 리서치.md 2.2 "약리학(PD) - 농도 -> 각성도 보정 계수"
2 import { sleepPressure, type SleepPressureSegment } from "./processS.js";
3
4 const GMAX_MIN = 0.05; /* 피로비율 0(안 피곤)일 때 최대 부스트. 근거 없는 근사치*/
5 const GMAX_MAX = 0.25; /* 피로비율 1(많이 피곤)일 때 최대 부스트. 근거 없는 근사치
6 const EC50 = 2; /* 반포화 농도(mg/L). 표준 200mg 섭취 시 최대혈중농도(4~6mg/L)의 절반 지점
7 const HILL_N = 1; /* Hill 계수. 협동결합 없다고 가정한 단순 포화 곡선
8 const H_MIN = 0.17; /* Process S의 H(t) 최솟값(가장 안 피곤)
9
10 function gmax(t: number, segment: SleepPressureSegment): number {
11     const fatigueRatio = (sleepPressure(t, segment) - H_MIN) / (1 - H_MIN);
12     return GMAX_MIN + (GMAX_MAX - GMAX_MIN) * fatigueRatio;
13 }
14
15 /** 카페인에 없으면(concentration=0) gPD(t)=1이라 P(t)=P0(t)로 자연스럽게 수렴한다. */
16 export function caffeineEffect(
17     t: number,
18     totalConcentration: number,
19     segment: SleepPressureSegment,
20 ): number {
21     const Gmax = gmax(t, segment);
22     const concentrationN = totalConcentration ** HILL_N;
23     return 1 + (Gmax * concentrationN) / (EC50 ** HILL_N + concentrationN);
24 }
```

최종 결합 — $P(t)$



baselineAlertness.ts · alertness.ts

$P_0(t) = (1-H(t)) + \kappa \cdot C(t) + \text{수면관성} + \text{오후슬럼프}$

$P(t) = P_0(t) \times gPD(t) \quad (\kappa = 0.2)$

왜 이렇게 작성했나

sleepPressure 는 ' 피로도 ' 라서 각성도로 쓰려면 뒤집어야 (1-H) 하는데 ,
처음엔 안 뒤집어서 ' 깨어있을수록 각성도가 높다 ' 는 반대 결과가 나온 걸 검증 중 발견해서 고쳤다 (코드 주석에 기록).

κ (일주기 진폭 계수)=0.2 는 원 논문값을 못 구해 근사치로 잡은 값이다 .
해당 계수는 유료 논문을 더 열람하면 값을 구할 수 있을 것으로 생각된다 .


```
server > src > calc > TS baselineAlertness.ts > ...
```

```
1  // 계산 모델 리서치.md 1장 "결합 공식( $P_0$ )" □ 카페인 없을 때 기본 각성도
2  import { sleepPressure, type SleepPressureSegment } from "./processS.js";
3  import { circadianRhythm } from "./processC.js";
4  import { sleepInertia } from "./sleepInertia.js";
5  import { lunchDip } from "./lunchDip.js";
6
7  const KAPPA = 0.2; // 일주기 진폭 계수. 논문 원값 미확보 □ 근사치(2026-07-13 결정)
8
9  export function baselineAlertness(t: number, segment: SleepPressureSegment): number {
10     // sleepPressure는 "피로도"(클수록 피곤함)라서, 각성도로 쓰려면 뒤집어야 함(1 - 피로도).
11     // 안 뒤집으면 "많이 깨어있을수록 각성도가 높다"는 반대 결과가 나오는 걸 검증 중 확인함(2026-07-13)
12     return (
13         (1 - sleepPressure(t, segment)) +
14         KAPPA * circadianRhythm(t) +
15         sleepInertia(t, segment) +
16         lunchDip(t)
17     );
18 }
```

server > src > calc > TS alertness.ts > ...

```
1  // 계산 모델 리서치.md 2.3 "최종 결합"  $P(t) = P_0(t) \times gPD(t)$ 
2  import { baselineAlertness } from "./baselineAlertness.js";
3  import { caffeineConcentration, type CaffeineDose } from "./caffeineConcentration.js";
4  import { caffeineEffect } from "./caffeineEffect.js";
5  import type { SleepPressureSegment } from "./processS.js";
6
7  export function alertness(
8    t: number,
9    segment: SleepPressureSegment,
10    doses: CaffeineDose[],
11    bodyWeightKg: number,
12    halfLifeHours: number,
13  ): number {
14    const totalConcentration = doses.reduce(
15      (sum, dose) => sum + caffeineConcentration(t, dose, bodyWeightKg, halfLifeHours),
16      0,
17    );
18
19    return baselineAlertness(t, segment) * caffeineEffect(t, totalConcentration, segment);
20  }
```

안전 섭취 한도



`dailyCaffeineLimit.ts` ·
`remainingCaffeineBudget.ts`

연령 · 건강상태별 한도 중 가장 낮은 값 적용

20 세 미만 : $\min(100\text{mg}, \text{체중} \times 2.5\text{mg/kg})$

왜 이렇게 작성했나

성인은 FDA 기준 고정 400mg 한도를 쓰지만,

12~19 세는 체중당 상한 (mg/kg) 권고가 일반적이라 나이 구간별로 계산 방식을 다르게 뒀다.

임신 · 심장질환 · 불안불면처럼 여러 조건이 겹치면, 그중 가장 엄격한 (가장 낮은) 한도 하나만 적용하도록 설계했다.

리액트 FE 에도 몸무게 입력란을 새로 설계했다.

```
server > src > calc > ts dailyCaffeineLimits.ts > ...
```

```
1 // 개발 일정.md 2주차 목요일 "안전 섭취 한도 로직(연령·건강상태별 한도)"
2 // 나이·체중·임신·심장질환·불안불면 여부를 보고 그날의 카페인 섭취 한도(mg)를 계산한다.
3 // 20세 미만 기준(100mg 또는 체중×2.5mg 중 낮은 값)은 기획서.md 6.3, 2026-07-15 결정으로 데이터 모델 스키마와 동기화.
4 export interface HealthProfile {
5   age: number;
6   weightKg: number;
7   pregnant: boolean;
8   heartCondition: boolean;
9   anxiety: boolean;
10 }
11
12 const ADULT_DEFAULT_MG = 400; // 건강한 성인 일반. 출처: FDA
13 const MINOR_MG = 100; // 12~19세 상한. 출처: AAP(미국소아과학회) 계열 권고
14 const MINOR_MG_PER_KG = 2.5; // 12~19세 체중당 상한(mg/kg). 기획서.md 6.3
15 const CHILD_MG = 0; // 12세 미만. 공개 자료 기준 섭취 자제 권고를 한도 0으로 반영
16 const RESTRICTED_CONDITION_MG = 200; // 임신·심장질환·불안불면 공통. 임신은 ACOG 기준, 심장질환·불안불면은 공식 고정
17
18 function ageBasedLimit(age: number, weightKg: number): number {
19   if (age < 12) return CHILD_MG;
20   if (age <= 19) return Math.min(MINOR_MG, MINOR_MG_PER_KG * weightKg);
21   return ADULT_DEFAULT_MG;
22 }
23
24 /** 여러 조건이 겹치면 그중 가장 엄격한(가장 낮은) 한도 하나만 적용한다. */
25 export function dailyCaffeineLimit(profile: HealthProfile): number {
26   const candidateLimits = [ageBasedLimit(profile.age, profile.weightKg)];
27   if (profile.pregnant) candidateLimits.push(RESTRICTED_CONDITION_MG);
28   if (profile.heartCondition) candidateLimits.push(RESTRICTED_CONDITION_MG);
29   if (profile.anxiety) candidateLimits.push(RESTRICTED_CONDITION_MG);
30
31   return Math.min(...candidateLimits);
32 }
```

server > src > calc > TS remainingCaffeineBudget.ts > ...

```
1  // 개발 일정.md 2주차 목요일 "오늘 섭취량 차감" 이 하루 한도에서 이미 마신 양을 뺀 나머지를 계산한다.
2  import type { CaffeineDose } from "../caffeineConcentration.js";
3
4  export function remainingCaffeineBudget(dailyLimitMg: number, todaysDoses: CaffeineDose[]): number {
5      const consumedMg = todaysDoses.reduce((sum, dose) => sum + dose.amountMg, 0);
6      return dailyLimitMg - consumedMg;
7  }
```

후보 그리드 탐색 — 목표 시각 역산



candidateSleepSegments.ts ·
singleExamScheduleSearch.ts

평소 / 원래 시각 ± 1 시간 , 15 분 간격 그리드에서
시험 시작 시각의 $P(t)$ 를 최대화하는 취침 · 기상 · 카페인 조합 탐색

왜 이렇게 작성했나

패널티 · 가중치 없는 단순 탐색으로 먼저 만들었다

— 아직 검증 안 된 S/C/ 카페인 함수 위에 무거운 최적화 로직까지 한 번에 얹으면 ,
버그가 생체수학 계산 문제인지 최적화 로직 문제인지 구분이 안 되기 때문이다 (2 주차 _ 계획 .md 3.2).

목표 시각은 여유시간을 빼지 않은 시험 시작 시각 그대로 쓴다 .

— 여유시간은 사람마다 편차가 커서 계산에 넣지 않고 화면 안내 문구로 대체 (2026-07-16 결정).

이 채점 구조를 3 주차 다중 시험 최적화가 그대로 재사용할 예정이다 .

```

1 // 개별 일정.md 2주차 수요일 "간단한 스크립트로 하루치 곡선 뽑아서 눈으로 검증" 작업용.
2 // 23시 취침, 7시 기상을 며칠 반복해서 정상상태(steady state)에 도달시킨 뒤,
3 // 마지막 하루 동안 15분 간격으로 P(t)(카페인 포함 최종 각성도)를 CSV로 뽑는다.
4 import { writeFileSync } from "node:fs";
5 import { alertness } from "../calc/alertness.js";
6 import { baselineAlertness } from "../calc/baselineAlertness.js";
7 import { sleepPressure, type SleepPressureSegment } from "../calc/processS.js";
8 import { caffeineConcentration, type CaffeineDose } from "../calc/caffeineConcentration.js";
9 import { caffeineEffect } from "../calc/caffeineEffect.js";
10 import { sensitivityToHalfLife } from "../calc/sensitivityToHalfLife.js";
11
12 const WAKE_HOUR = 7;
13 const SLEEP_HOUR = 23;
14 const WARMUP_DAYS = 3; // 정상상태로 수렴시키기 위한 예열 일수
15
16 function buildSegments(days: number): SleepPressureSegment[] {
17   const segments: SleepPressureSegment[] = [];
18   // 예열 시작점: 첫 기상 시각(H_MIN 근처에서 출발한다고 가정)
19   let startTime = WAKE_HOUR - 24 * WARMUP_DAYS;
20   let startPressure = 0.17; // Process S의 최솟값(H_MIN)에서 출발
21   let isAsleep = false;
22
23   const totalSegments = (days + WARMUP_DAYS) * 2;
24   for (let i = 0; i < totalSegments; i++) {
25     const segment: SleepPressureSegment = { startTime, startPressure, isAsleep };
26     segments.push(segment);
27
28     const duration = isAsleep ? WAKE_HOUR + 24 - SLEEP_HOUR : SLEEP_HOUR - WAKE_HOUR;
29     const nextTime = startTime + duration;
30     const nextPressure = sleepPressure(nextTime, segment);
31
32     startTime = nextTime;
33     startPressure = nextPressure;
34     isAsleep = !isAsleep;
35   }
36   return segments;
37 }

```

```

39 function segmentAt(segments: SleepPressureSegment[], t: number): SleepPressureSegment {
40   let current = segments[0];
41   for (const segment of segments) {
42     if (segment.startTime > t) break;
43     current = segment;
44   }
45   return current;
46 }
47
48 const segments = buildSegments(1);
49
50 // 검증용 예시 시나리오: 체중 65kg, 카페인 민감도 "보통", 08시 커피 200mg + 13시 에너지드링크 150mg
51 const BODY_WEIGHT_KG = 65;
52 const HALF_LIFE_HOURS = sensitivityToHalfLife("보통");
53 const DOSES: CaffeineDose[] = [
54   { time: WAKE_HOUR + 1, amountMg: 200 }, // 08:00 커피
55   { time: 13, amountMg: 150 }, // 13:00 에너지드링크
56 ];
57
58 const rows: string[] = [
59   "time,hour,H_sleepPressure,P0_baseline,concentration_mgL,gPD_effect,P_final",
60 ];
61
62 for (let t = 0; t <= 24; t += 0.25) {
63   const segment = segmentAt(segments, t);
64   const H = sleepPressure(t, segment);
65   const P0 = baselineAlertness(t, segment);
66   const concentration = DOSES.reduce(
67     (sum, dose) => sum + caffeineConcentration(t, dose, BODY_WEIGHT_KG, HALF_LIFE_HOURS),
68     0,
69   );
70   const gPD = caffeineEffect(t, concentration, segment);
71   const P = alertness(t, segment, DOSES, BODY_WEIGHT_KG, HALF_LIFE_HOURS);
72
73   const hourLabel = `${String(Math.floor(t) % 24).padStart(2, "0")}:${t % 1 === 0 ? "00" : "15"}`;
74   rows.push(
75     [t.toFixed(2), hourLabel, H.toFixed(4), P0.toFixed(4), concentration.toFixed(4), gPD.toFixed(4), P.toFixed(4)]
76   );
77 }
78
79 const outputPath = process.argv[2] ?? `./tmp/alertness-curve.csv`;
80 writeFileSync(outputPath, rows.join("\n"), "utf-8");
81 console.log(`검증용 CSV 저장 완료: ${outputPath} (${rows.length - 1}개 시점)`);

```



```

1 // 2주차 목표일 "목표 각성 시작 시간" 검증용 1 평소 스케줄 그대로 쓴 점수와
2 // 그리드 탐색으로 찾은 최적 스케줄의 점수를 비교해서, 탐색이 실제로 더 나은(또는 최소한 같은) 조합을
3 // 찾는지 눈으로 확인한다. 목표 시작은 여유시간을 빼지 않은 시험 시작 시작 그대로 쓴다(2026-07-16 결정).
4 import { alertness } from "../calc/alertness.js";
5 import { buildCandidateSegments, segmentAt } from "../calc/candidateSleepSegments.js";
6 import { sensitivityToHalfLife } from "../calc/sensitivityToHalfLife.js";
7 import { findBestSingleExamSchedule } from "../calc/singleExamScheduleSearch.js";
8
9 const HABITUAL_BED_TIME = 23;
10 const HABITUAL_WAKE_TIME = 7;
11 const BODY_WEIGHT_KG = 65;
12 const HALF_LIFE_HOURS = sensitivityToHalfLife("보통");
13 const EXAM_START_TIME = 9; // 09:00 시험
14 const PLANNED_DOSES = [{ time: 8, amountMg: 200 }]; // 08:00 커피 200mg
15
16 // 평소 스케줄(오프셋 0) 그대로 썼을 때의 점수
17 const habitualSegments = buildCandidateSegments(
18   HABITUAL_BED_TIME,
19   HABITUAL_WAKE_TIME,
20   HABITUAL_BED_TIME,
21   HABITUAL_WAKE_TIME,
22 );
23 const habitualSegmentAtExam = segmentAt(habitualSegments, EXAM_START_TIME);
24 const habitualScore = alertness(EXAM_START_TIME, habitualSegmentAtExam, PLANNED_DOSES, BODY_WEIGHT_KG, HALF_LI
25
26 const best = findBestSingleExamSchedule({
27   habitualBedTime: HABITUAL_BED_TIME,
28   habitualWakeTime: HABITUAL_WAKE_TIME,
29   plannedDoses: PLANNED_DOSES,
30   examStartTime: EXAM_START_TIME,
31   bodyWeightKg: BODY_WEIGHT_KG,
32   halfLifeHours: HALF_LIFE_HOURS,
33 });
34
35 console.log("시험 시작 시작(목표):", EXAM_START_TIME.toFixed(2));
36 console.log("평소 스케줄 그대로 점수(habitual score):", habitualScore.toFixed(4));
37 console.log("탐색으로 찾은 최적 스케줄:", {
38   bedTime: best.bedTime,
39   wakeTime: best.wakeTime,
40   doses: best.doses,
41   score: Number(best.score.toFixed(4)),
42 });
43 console.log(
44   best.score >= habitualScore
45     ? "[OK] 탐색 결과가 평소 스케줄보다 같거나 높음"
46     : "[FAIL] 탐색 결과가 평소 스케줄보다 낮음 1 버그 의심",
47 );

```

이번 주 핵심 결과 — 목표 각성 시각 역산

예시 : 체중 65kg · 카페인 민감도 " 보통 " · 08 시 커피 200mg · 09 시 시험 — 목표 시각은 09 시 (시험 시작) 그대로

평소 스케줄 그대로

0.7196

시험 시작 시각 $P(t)$ 점수

취침 23:00 · 기상 07:00



탐색으로 찾은 최적 스케줄

0.7486

시험 시작 시각 $P(t)$ 점수 (+4.0%)

취침 22:00 · 기상 07:30

검증 스크립트 (`verifySingleExamSchedule.ts`) 로 확인 — 취침 · 기상 · 카페인 후보를 ± 1 시간 그리드에서 탐색한 결과,
1 시간 더 일찍 자고 30 분 늦게 일어나는 조합이 시험 시작 시각의 각성도를 더 높인다는 걸 수치로 확인했다.

이번 주 의사결정 · 이슈 · 버그

- 1 카페인 근사 모델 확정 — 표준 포화형 곡선 (Hill $n=1$) 으로 근사, G_{max} 는 근거 없는 근사치로 명시
- 2 검증 중 발견한 보정 항 추가 — 수면관성 · 오후슬럼프 dips 을 원 수식에 보완
- 3 sleepPressure 부호 버그 수정 — 각성도 계산 시 뒤집어야 함을 검증 스크립트로 확인 후 수정
- 4 목표 각성 시각에서 이동시간 제외 — 통학 방식이 제각각이라 일반화 어려워 안내 문구로 대체
- 5 후보 탐색 그리드 확정 — ± 1 시간 · 15 분 간격, 카페인은 시각만 후보로 흔들고 용량은 고정
- 6 여유시간도 계산에서 제외 — 목표는 그냥 " 시험 시작 시각에 각성도 최고 ", 안내 문구만 유지

2 주차 · 이슈 정리

3 주차로 넘어가는 것 — GitHub 마일스톤 정리

GitHub 마일스톤 "2 주차"에 남아있던 이슈 3 개를 오늘 정리했다



#8

목표 작성 시각 역산 (단순 탐색 , 시험 1 개)

완료 — 닫음



#10

다중 시험 최적화 — 그리드 결정변수 정의

3 주차로 이동



#14

Supabase 프로젝트 생성 및 마이그레이션

3 주차로 이동

#10 · #14 는 원래 3 주차 범위 (다중 시험 최적화 · Supabase 연동) 에 속하는 작업이라 마일스톤만 옮겼고,
#8 은 오늘 `candidateSleepSegments.ts` + `singleExamScheduleSearch.ts` 로 실제 구현을 마쳐서 닫았다.

다음 주 예고 · 고민되는 점



3 주차 목표

"계산하기" 누르면 진짜
계산해서 결과가 나오는
상태를 만드는 게 목표.
다중 시험 최적화 + API +
화면 연동까지 한 번에



가장 큰 과제

시험 1개용 그리드 탐색을
여러 시험 동시 배분으로
확장해야 하는데, 패널티·
가중치 설계를 어떻게 할지
아직 구체화 안 됨



아직 안 정한 것들

카페인 mg 직접 입력 폼,
스케줄 조정 화면 날짜 선택
UI, 공통 컴포넌트 (Chart 등)
분리 범위 — spec.md "열려
있는 질문"에 정리해둬م